

DOSSIER PROFESSIONNEL (DP)

Nom de naissance

□ LECART

Prénom

□ Christophe

Adresse

□ 56 grande rue 76510 NOTRE DAME
D'ALIERMONT

Titre professionnel visé

Concepteur Développeur d'Applications

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer une application sécurisée	p.5
AT1 n°1 : Authentification JWT, cookies HTTP-only, middleware Next.js	p.5
AT1 n°2 : Validation des données et protection des routes API avec Zod	p.7
AT1 n°3 : Système de réactions universel posts/stories/commentaires en temps réel	p.8
Concevoir et développer une application sécurisée organisée en couches	p.10
AT2 n°1 : Architecture full-stack en couches et schéma Prisma (18 modèles)	p.10
AT2 n°2 : Messagerie temps réel — Server-Sent Events et Upstash Redis	p.12
AT2 n°3 : Design system dark mode, thème provider et composant multi-médias	p.14
Préparer le déploiement d'une application sécurisée	p.16
AT3 n°1 : Conteneurisation Docker multi-stage et déploiement Vercel (intégration Git native)	p.16
AT3 n°2 : Déploiement en production Vercel avec PostgreSQL Neon et Prisma	p.18
AT3 n°3 : Monitoring, observabilité et documentation opérationnelle	p.20
Titres, diplômes, CQP, attestations de formation <i>(facultatif)</i>	p.22
Déclaration sur l'honneur	p.23
Documents illustrant la pratique professionnelle <i>(facultatif)</i>	p.24
Annexes <i>(Si le RC le prévoit)</i>	p.28

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type

1

Développer une application sécurisée

AT1 n°1

Authentification JWT, cookies HTTP-only, middleware Next.js

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet Social Network (baptisé **Konekt** par l'équipe) développé à Zone01, j'ai pris en charge l'intégralité du **système d'authentification**. La contrainte principale était l'hébergement sur Vercel (infrastructure serverless) : pas de serveur persistant, donc pas de session côté serveur classique — toute la sécurité devait reposer sur des mécanismes compatibles Edge Runtime.

J'ai conçu **trois flux distincts**.

- **Le flux classique email/mot de passe :**

A l'inscription, le mot de passe est haché via bcrypt (facteur de coût 12) avant toute persistance. A la connexion, un JWT signé (HS256, bibliothèque jose) est émis et placé dans un cookie httpOnly — invisible du JavaScript, résistant aux attaques XSS. Les messages d'erreur restent volontairement génériques pour ne pas révéler l'existence d'un compte.

- **Le flux OAuth Google :** l'utilisateur est redirigé avec un paramètre state anti-CSRF ; au retour, le state est vérifié, le profil Google récupéré, l'utilisateur créé ou mis à jour en base (upsert Prisma), puis un JWT local émis.
- **Le flux de déconnexion :** le cookie est expiré côté client (expires: new Date(0)). La protection des routes est assurée par un middleware Next.js (src/middleware.ts) : il vérifie le JWT, extrait le userId et l'injecte dans le header x-user-id pour les handlers downstream. Un rate limiting distribué (Upstash Redis, 5 tentatives/60s par IP) protège contre les attaques par force brute.

Extrait — génération JWT et placement en cookie :

```
// src/lib/jwt/signJwt.ts
export async function signJwt(payload: any, expiresIn = JWT_EXPIRATION.DEFAULT) {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })
    .setIssuedAt()
    .setExpirationTime(expiresIn)
    .sign(new TextEncoder().encode(process.env.JWT_SECRET!));
}

// src/app/api/public/auth/login/route.ts
const token = await login(email, password);
const res = NextResponse.json({ success: true });
res.cookies.set("authToken", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "lax",
  maxAge: 60 * 60 * 24 * 7,
  path: "/",
});
return res;
```

2. Précisez les moyens utilisés :

- jose — JWT compatible Edge Runtime (SignJWT HS256, jwtVerify)
- bcrypt — hachage mot de passe, cost factor 12
- Cookie authToken — HTTP-only, Secure (prod), SameSite=Lax, maxAge 7 jours — stockage sécurisé du JWT côté client
- @upstash/ratelimit + @upstash/redis — rate limiting serverless sliding window 5 req/60s
- Prisma ORM — findUnique (login), upsert (OAuth Google)
- Next.js 15 middleware Edge Runtime — interception de toutes les requêtes, injection header x-user-id
- Variables d'environnement Vercel — JWT_SECRET, GOOGLE_CLIENT_ID/SECRET, UPSTASH_*

3. Avec qui avez-vous travaillé ?

Le développement a été réalisé en autonomie au sein d'une équipe de 5 développeurs.

Les choix architecturaux (JWT HTTP-only vs localStorage, stateless vs sessions Redis) ont été présentés et validés collectivement.

Revue de code par les pairs sur les pull requests liées à la sécurité.

4. Contexte

Nom de l'entreprise, organisme ou association ▯

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service ▯ Projet Social Network — Authentification & Sécurité

Période d'exercice ▯ Du 26/05/2025 au 18/09/2025

5. Informations complémentaires (facultatif)

Cookie HTTP-only vs localStorage : le localStorage est accessible depuis JavaScript et vulnérable aux attaques XSS.

Un cookie HTTP-only est invisible du JavaScript, même en cas d'injection de code malveillant.

L'attribut SameSite=Lax protège contre le CSRF en bloquant l'envoi du cookie depuis des sites tiers.

Ces choix sont conformes à l'OWASP Authentication Cheat Sheet (A07:2021).

Point d'amélioration : l'architecture stateless ne permet pas de révoquer un token avant son expiration. Une liste noire Redis permettrait une révocation immédiate en cas de compromission (axe d'amélioration identifié).

Activité-type

1

AT1 n°2

Développer une application sécurisée

Validation des données et protection des routes API avec Zod

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En complément de l'authentification, j'ai mis en place une **stratégie de validation stricte sur l'ensemble des routes API**. Chaque endpoint expose un contrat de données : les requêtes entrantes sont validées via Zod avant tout traitement métier. 36 schémas Zod couvrent tous les endpoints (création de post, envoi de message, inscription, réactions, groupes, événements).

Le middleware Next.js intercepte toutes les routes `/api/private/**` et rejette les requêtes non authentifiées. Des helpers de réponse standardisés (400 avec détail des champs invalides, 401, 403, 404) garantissent une API cohérente pour le front-end. La validation isomorphique Zod permet de réutiliser les mêmes schémas côté client pour la validation des formulaires, sans duplication de logique.

2. Précisez les moyens utilisés :

- Zod (TypeScript-first) — `safeParse`, `z.infer<typeof schema>`, `flatten().fieldErrors`
- Next.js middleware Edge Runtime — header `x-user-id` injecté, rejet 401 des requêtes non authentifiées
- TypeScript strict — types inférés depuis les schémas Zod, zéro cast manuel
- Helpers de réponse API standardisés — `src/lib/server/api/response.ts`

3. Avec qui avez-vous travaillé ?

Nous nous sommes coordonnés avec les 5 membres de l'équipe pour aligner les structures de données front/back.

Chaque schéma Zod servait de contrat partagé : les développeurs *front* pouvaient l'importer directement pour valider les formulaires.

Revue de code sur la cohérence des types entre schémas Zod et modèles Prisma.

4. Contexte

Nom de l'entreprise, organisme ou association

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

Projet Social Network — Couche API & Sécurité des données

Période d'exercice

Du 26/05/2025 au 18/09/2025

5. Informations complémentaires (facultatif)

Validation isomorphique avec Zod : l'avantage principal est l'inférence de types TypeScript — `z.infer<typeof schema>` qui génère automatiquement le type correspondant. Le même schéma sert à la fois de validateur runtime et de source de type statique, éliminant toute désynchronisation entre validation et typage. Le `safeParse` retourne un objet discriminé (`success/error`) exploitable sans `try/catch`, ce qui rend la gestion d'erreur explicite et prévisible.

Activité-type

1

Développer une application sécurisée

AT1 n°3

Système de réactions universel posts/stories/commentaires en temps réel

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le système de réactions est ma contribution personnelle la plus complète : de la conception du modèle de données jusqu'au composant React final, en passant par 5 routes API CRUD et un hook React Query. Le système supporte 7 types de réactions (7 types : LIKE, DISLIKE, LOVE, LAUGH, SAD, ANGRY, WOW") sur 3 entités distinctes (Post, Story, Comment) avec comptage en temps réel via Server-Sent Events.

- Modèle Prisma : table Reaction polymorphique avec champ type enum, contrainte unique (userId, postId/storyId/commentId) pour empêcher le double vote, groupBy pour agréger les counts par type.
- API : upsert pour basculer entre types de réaction, suppression atomique à la déconnexion de la réaction.
- Front-end : composant ReactionPicker avec animation hover, mise à jour optimiste via React Query (l'UI reflète le changement avant la réponse serveur), synchronisation SSE pour afficher en temps réel les réactions des autres utilisateurs.

2. Précisez les moyens utilisés :

- Prisma ORM — modèle Reaction polymorphique, upsert, groupBy pour les counts par type
- Next.js Route Handlers — CRUD complet (GET, POST, PUT, DELETE)
- React Query — mise à jour optimiste, invalidation de cache ciblée
- Server-Sent Events — synchronisation temps réel des nouvelles réactions
- Tailwind CSS — positionnement popup, animations hover

3. Avec qui avez-vous travaillé ?

Le développement a été réalisé entièrement en autonomie — 11 commits entre le 8 et le 28 juillet 2025.

Le modèle de données a été présenté à l'équipe avant implémentation pour valider l'approche polymorphique.

L'ensemble final a été revu par un pair avant merge sur la branche principale.

4. Contexte

Nom de l'entreprise, organisme ou association □

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

□ Projet Social Network — Système de réactions universel

Période d'exercice

□ Du 26/05/2025 au 18/09/2025

5. Informations complémentaires (*facultatif*)

Justification du modèle polymorphique : trois architectures étaient envisageables :

(1) trois tables séparées (PostReaction, StoryReaction, CommentReaction) : duplication de logique métier et de routes API

(2) table unique avec colonnes nullable : requêtes complexes, contraintes d'intégrité difficiles à garantir

(3) table unique polymorphique avec FK nullable — retenue pour son équilibre entre flexibilité et lisibilité.

La contrainte unique composite garantit l'idempotence des votes : upsert sur (userId + entitéId) remplace sans créer de doublon.

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

AT2 n°1 Architecture full-stack en couches et schéma Prisma (18 modèles)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai participé à la conception de l'architecture du projet et pris en charge la **modélisation complète du schéma de données Prisma**.

La contrainte principale était de supporter un réseau social complet (posts, stories, messagerie, groupes, événements, notifications) avec une base de données unique cohérente.

L'architecture retenue suit un découpage en couches strict : présentation (React Server/Client Components), API (Next.js Route Handlers), service (src/lib/server/), données (Prisma + PostgreSQL).

Le schéma Prisma couvre 18 modèles : User, Post, Comment, Reaction, Story, Message, Conversation, ConversationMember, GroupMessage, GroupMember, GroupInvitation, GroupJoinRequest, Event, Rsvp, Notification, UserSettings, Friendship, Account. Chaque relation (1-N, N-N avec table pivot) est typée et documentée. Les migrations Prisma (migrate dev/deploy) versionnent l'évolution du schéma tout au long du projet, permettant des rollbacks contrôlés.

2. Précisez les moyens utilisés :

- Prisma ORM 6.x — schéma déclaratif, migrate dev/deploy, client TypeScript généré automatiquement
- PostgreSQL 15 (Neon serverless) — relations, index, contraintes d'intégrité référentielle
- Next.js 15 App Router — Server Components, Route Handlers, layouts imbriqués
- TypeScript strict — types inférés depuis le client Prisma, zéro cast manuel
- draw.io — MCD et diagramme de flux pour la documentation technique

3. Avec qui avez-vous travaillé ?

Modélisation collective impliquant les 5 développeurs : chaque membre apportait ses besoins fonctionnels (messagerie, groupes, événements, stories) et le schéma résultait d'itérations collectives.

J'ai animé les sessions de modélisation et maintenu le schéma Prisma comme référence unique partagée par toute l'équipe.

4. Contexte

Nom de l'entreprise, organisme ou association Zone 01 ROUEN NORMANDIE

Chantier, atelier, service Projet Social Network — Architecture & Modélisation

Période d'exercice Du 26/05/2025 au 18/09/2025

5. Informations complémentaires *(facultatif)*

Server Components vs Client Components : Next.js 15 App Router distingue les Server Components (exécutés côté serveur, accès direct à la base de données, zéro JavaScript envoyé au client) et les Client Components (interactivité, hooks React).

Règle appliquée dans le projet : Server Component par défaut, Client Component uniquement quand l'interactivité ou les hooks (useState, useEffect) sont nécessaires.

Cette séparation réduit le bundle JavaScript envoyé au navigateur et améliore les Core Web Vitals (LCP notamment).

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

AT2 n°2

Messagerie temps réel — Server-Sent Events et Upstash Redis

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et développé le **système de messagerie instantanée en temps réel**, en résolvant une contrainte d'infrastructure majeure : Vercel exécute les route handlers comme des fonctions serverless sans état persistant, rendant les WebSockets impossibles. La solution retenue : Server-Sent Events combinés à Upstash Redis comme bus de messages.

Architecture :

L'émetteur (Alice) POST /api/private/chat/send — le message est persisté en base via Prisma et publié dans une clé Redis temporaire (SET avec TTL 60s).

Le récepteur (Bob) maintient une connexion SSE longue durée sur /api/private/chat/listen — le handler effectue un polling GET toutes les 500ms sur sa clé Redis, avec déduplication via lastMessageId : si l'identifiant du message lu est identique à la lecture précédente, il n'est pas renvoyé.

Quand un message nouveau est trouvé, il est immédiatement envoyé dans le flux SSE vers le client.

2. Précisez les moyens utilisés :

- Server-Sent Events — flux unidirectionnel serveur → client, ReadableStream compatible Edge Runtime
- Upstash Redis — SET avec TTL 60s, GET toutes les 500ms, déduplication via lastMessageId compatible serverless
- Prisma ORM — persistance des messages en PostgreSQL
- EventSource (navigateur) — reconnexion automatique intégrée
- Polling 500ms — compromis latence (~250ms en moyenne) / charge serveur

3. Avec qui avez-vous travaillé ?

Nous avons travaillé en binôme sur la partie temps réel.

Revues portant sur la gestion des cas limites : messages en rafale, destinataire déconnecté, expiration de clé Redis avant lecture, reconnexion EventSource.

La décision de déduplication via lastMessageId plutôt qu'une suppression Redis a été validée en binôme : ce mécanisme préserve les messages en cas de reconnexion SSE.

4. Contexte

Nom de l'entreprise, organisme ou association

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

Projet Social Network — Messagerie temps réel

Période d'exercice

Du 26/05/2025 au 18/09/2025

5. Informations complémentaires (facultatif)

Pourquoi SSE plutôt que WebSockets ?

Vercel exécute les route handlers comme des fonctions serverless à durée limitée.

WebSocket requiert une connexion TCP persistante et bidirectionnelle incompatible avec ce modèle d'exécution.

SSE est du HTTP standard (port 80/443, compatible avec tous les proxies et CDN), unidirectionnel serveur → client, avec reconnexion automatique intégrée dans l'API EventSource du navigateur.

Le canal client → serveur reste un POST HTTP classique. Cette architecture est compatible Edge Runtime et scalable horizontalement sans serveur dédié.

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

AT2 n°3 □ Design system dark mode, thème provider et composant de création de posts multi-médias

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai défini l'identité visuelle du projet **Social Network** (Konekt) et développé le **composant de création de posts multi-médias**, utilisé par toute l'équipe comme base pour les *features* groupes et événements.

- Design system : définition des design tokens (palette de couleurs en CSS Custom Properties, typographie, espacements) dans `globals.css`, adoptés par toute l'équipe comme référence unique.
- Dark mode : intégration de `next-themes` avec `ThemeProvider` englobant l'application, bascule persistée en `localStorage`, SSR-safe — la classe thème est injectée avant le premier rendu pour éviter le flash de contenu non stylé (FOUC).
- Composant `PostCreator` : upload multi-médias (images, vidéos) avec prévisualisation avant envoi, éditeur de texte avec détection de mention `@user` et hashtag `#tag`, sélecteur d'audience (public/amis/groupe), validation client `Zod` avant envoi, gestion des erreurs d'upload `Cloudinary` avec feedback utilisateur.

2. Précisez les moyens utilisés :

- CSS Custom Properties — design tokens pour la palette, point de modification unique pour toute l'équipe
- `next-themes` — dark/light mode SSR-safe, persistance `localStorage`, `ThemeProvider`
- `Cloudinary` — upload et transformation d'images/vidéos, URLs optimisées
- `React Hook Form` + `Zod` — validation côté client, gestion des erreurs par champ
- `Tailwind CSS` — utility classes basées sur les tokens définis

3. Avec qui avez-vous travaillé ?

Le Design system a été défini en autonomie en mai 2025, puis adopté par toute l'équipe.

La documentation des variables CSS a été partagée via le README du projet.

Le composant `PostCreator` a été intégré par d'autres membres dans leurs *features* (groupes, événements) sans modification nécessaire, ce qui valide la robustesse de l'interface proposée.

4. Contexte

Nom de l'entreprise, organisme ou association □

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

Projet Social Network — Design system & Composants UI

Période d'exercice □ Du

26/05/2025 au 18/09/2025

5. Informations complémentaires *(facultatif)*

Importance du design system en équipe : sans tokens de couleurs partagés, chaque développeur utilise ses propres valeurs hexadécimales, rendant le dark mode impossible à implémenter de manière cohérente.

Les CSS Custom Properties permettent de changer toute la palette en un seul fichier.

next-themes résout le problème de flash (FOUC) en injectant la classe thème avant le premier rendu côté serveur, grâce à un script inline dans le head — ce qui est impossible avec une approche localStorage pure côté client.

Activité-type

3

AT3 n°1

Préparer le déploiement d'une application sécurisée

Containerisation Docker multi-stage et déploiement Vercel

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et mis en place **la chaîne de build du projet** : Dockerfile multi-stage optimisé et docker-compose.yml avec profils dev/prod.

- Le Dockerfile sépare l'étape builder (bun install, prisma generate, next build) de l'image runner finale, faisant passer l'image de plus de 2 Go à moins de 500 Mo.
- Le docker-compose orchestre uniquement le service applicatif — la base de données Neon et Redis Upstash sont des services externes.
- Le déploiement en production est effectué via la CLI Vercel avec prisma migrate deploy avant chaque mise en ligne. L'analyse qualité est assurée par SonarQube.

2. Précisez les moyens utilisés :

- Docker multi-stage build — séparation build/runtime, image production allégée
- Docker Compose — orchestration locale, service Next.js uniquement (PostgreSQL Neon et Redis Upstash sont des services externes)
- Bun — gestionnaire de paquets et runtime (image de base oven/bun:1)
- Vercel CLI
- Prisma — migrate deploy en pré-déploiement Vercel

3. Avec qui avez-vous travaillé ?

Le travail a été réalisé en collaboration pour la stabilisation de la chaîne CI/CD.

La PR #118 représente le point de convergence après plusieurs itérations : dépendances système manquantes dans l'image Docker (openssl pour Prisma), variables d'environnement absentes en CI, migrations Prisma échouant sur la base de test.

Chaque échec de pipeline était documenté et corrigé en binôme avant le merge final.

4. Contexte

Nom de l'entreprise, organisme ou association

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

Projet Social Network — Infrastructure & CI/CD

Période d'exercice

Du

26/05/2025

au

18/09/2025

5. Informations complémentaires (*facultatif*)

Optimisation de l'image Docker avec le multi-stage : l'image builder contient toutes les devDependencies (TypeScript, ESLint, Prisma CLI, types) nécessaires à la compilation mais inutiles en production.

L'étape runner copie uniquement les artefacts compilés (.next/), les node_modules de production et le binaire Prisma généré.

Résultat : image production inférieure à 500 Mo, temps de démarrage du conteneur réduit, surface d'attaque réduite (pas de compilateur ni d'outils de développement dans le conteneur de production).

Activité-type

3

AT3 n°2

Préparer le déploiement d'une application sécurisée

Déploiement en production Vercel avec PostgreSQL Neon et migrations Prisma

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai configuré et documenté le **déploiement en production** sur Vercel, avec PostgreSQL Neon comme base de données et Prisma pour la gestion des migrations.

- Vercel : configuration de toutes les variables d'environnement (JWT_SECRET, DATABASE_URL, DIRECT_URL, GOOGLE_CLIENT_ID/SECRET, CLOUDINARY_*, UPSTASH_REDIS_REST_URL/TOKEN), séparation des environnements Preview et Production, headers de sécurité configurés dans next.config.ts (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options).
- PostgreSQL Neon : deux URLs sont nécessaires — DATABASE_URL pointe vers le pooler pgBouncer pour toutes les requêtes applicatives Prisma, DIRECT_URL pointe vers la connexion PostgreSQL directe utilisée exclusivement pour les migrations (pgBouncer en mode transaction pooling ne supporte pas les commandes DDL).
- Les migrations Prisma sont exécutées automatiquement en post-build hook via prisma migrate deploy. Tests de non-régression post-déploiement : connexion, création de post, messagerie temps réel, cookies HTTP-only visibles dans DevTools, redirections middleware vers /login.

2. Précisez les moyens utilisés :

- Vercel — plateforme Next.js native, secrets chiffrés, environnements Preview/Production séparés
- PostgreSQL Neon — serverless, pgBouncer (pooler), connexion directe pour les migrations
- Prisma migrate deploy — migrations reproductibles et versionnées en production
- next.config.ts — headers de sécurité CSP, X-Frame-Options, X-Content-Type-Options

3. Avec qui avez-vous travaillé ?

Configuration collective : Prisma/Neon configurés par moi-même, OAuth Google et Cloudinary configurés par d'autres membres de l'équipe.

Nous nous sommes coordonnés sur les variables d'environnement partagées pour éviter les conflits.

Revue de la configuration Vercel par le groupe avant le passage en production.

4. Contexte

Nom de l'entreprise, organisme ou association

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service

Projet Social Network — Mise en production

Période d'exercice

Du 26/05/2025 au 18/09/2025

5. Informations complémentaires *(facultatif)*

Pourquoi deux URLs pour Neon :

Neon est une base de données PostgreSQL serverless qui utilise pgBouncer comme pooler de connexions pour absorber les pics de charge des fonctions serverless. Ce pooler fonctionne en mode transaction pooling, qui ne supporte pas les commandes DDL (CREATE TABLE, ALTER TABLE, DROP) utilisées par les migrations Prisma. DATABASE_URL pointe donc vers le pooler pour toutes les requêtes applicatives, DIRECT_URL pointe vers la connexion PostgreSQL directe pour les migrations uniquement.

Prisma utilise automatiquement la bonne URL selon l'opération effectuée.

Activité-type 3

AT3 n°3

Préparer le déploiement d'une application sécurisée

Monitoring, observabilité et documentation opérationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai défini et documenté la **stratégie d'observabilité et les procédures opérationnelles** du projet Social Network (Konekt), afin que n'importe quel membre de l'équipe puisse gérer un déploiement ou diagnostiquer un incident sans dépendance à une seule personne.

- Monitoring Vercel : suivi des Core Web Vitals (LCP, FID, CLS) via Vercel Analytics avec tableau de bord dédié, consultation des logs des fonctions serverless en temps réel, alertes configurées sur les dépassements de quota (bandwidth, invocations). Logs applicatifs : console.error structuré avec contexte (route, userId tronqué) dans tous les routes handlers, sans exposition de données sensibles dans les logs Vercel accessibles à l'équipe.
- Documentation opérationnelle rédigée et partagée : procédure de déploiement step-by-step (variables d'environnement requises, ordre des étapes, vérifications post-déploiement), procédure de rollback application via vercel rollback pour retour instantané à un déploiement précédent, procédure de rollback base de données via migration inverse Prisma (manuel, nécessite une fenêtre de maintenance). Distinction explicite entre les deux types de rollback car leurs délais et impacts sont radicalement différents.

2. Précisez les moyens utilisés :

- Vercel Analytics — Core Web Vitals en production, tableau de bord LCP/FID/CLS
- Vercel Logs — logs des fonctions serverless, erreurs en temps réel
- console.error structuré — contexte route/userId sans données sensibles
- prisma migrate resolve — résolution manuelle des migrations en échec
- vercel rollback — retour instantané à un déploiement précédent

3. Avec qui avez-vous travaillé ?

La documentation et la stratégie de monitoring ont été rédigées en autonomie puis présentées à l'équipe lors d'une session de partage des connaissances en fin de projet.

Elles sont utilisées par les autres membres pour les déploiements suivants sans intervention de ma part, ce qui valide leur clarté et leur complétude.

4. Contexte

Nom de l'entreprise, organisme ou association □

Zone 01 ROUEN NORMANDIE

Chantier, atelier, service □

Projet Social Network — Monitoring & Documentation opérationnelle

Période d'exercice □

Du 26/05/2025 au 18/09/2025

5. Informations complémentaires *(facultatif)*

Documentation opérationnelle : valeur pour l'équipe et pour la certification.

Un déploiement réussi ne se limite pas à mettre le code en production. La documentation des procédures (déploiement, rollback, gestion des secrets, résolution d'incidents) permet à n'importe quel membre d'intervenir en cas de problème, sans dépendance à une seule personne.

Elle constitue également une preuve de maturité opérationnelle pour l'évaluation RNCP : la compétence "préparer le déploiement" ne se limite pas à écrire un Dockerfile, elle inclut la capacité à maintenir et à transmettre les procédures qui garantissent la continuité du service.

DOSSIER PROFESSIONNEL (DP)



Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Concepteur Développeur d'Applications	Zone01 Rouen	Juin 2024 – juin 2026
BTS MAI	CFAICF Elbeuf	Sept 2001 – juin 2003
BAC PRO MSMA	CFAI La Châtaigneraie / Mesnil-Esnard	Sept 1999 - juin 2001
BEP MSMA	Lycée Bernard Palissy / Maromme	Sept 1997 – juin 1999

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] ***Christophe LECART***,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à *NOTRE DAME D'ALIERMONT* le *22/05/2026*
pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Dictionnaire de données :

<https://github.com/CLecart/social-network/blob/docs/dossier-certification/dossier-certification/03-conception/donnees.md>

Cahier des Charges :

<https://github.com/CLecart/social-network/blob/main/dossier-certification/02-cahier-des-charges/README.md>

Code — Route auth JWT + cookie :

<https://github.com/arocchet/social-network/blob/main/src/app/api/public/auth/login/route.ts>

Code — Middleware protection des routes :

<https://github.com/arocchet/social-network/blob/main/src/middleware.ts>

Code — Schéma Prisma 18 modèles :

<https://github.com/arocchet/social-network/blob/main/prisma/schema.prisma>

Code — SSE messagerie temps réel :

<https://github.com/arocchet/social-network/blob/main/src/app/api/private/chat/listen/route.ts>

Code — Dockerfile multi-stage:

<https://github.com/arocchet/social-network/blob/main/Dockerfile>

Commit — auth login/register/middleware:

<https://github.com/arocchet/social-network/commit/f08fbee>

Commit — fix(ratelimit):

<https://github.com/arocchet/social-network/commit/c27bb60>

Commit — feat reactions universelles:

<https://github.com/arocchet/social-network/commit/158d8ee>

Commit — fix real-time reactions:

<https://github.com/arocchet/social-network/commit/0ca9588>

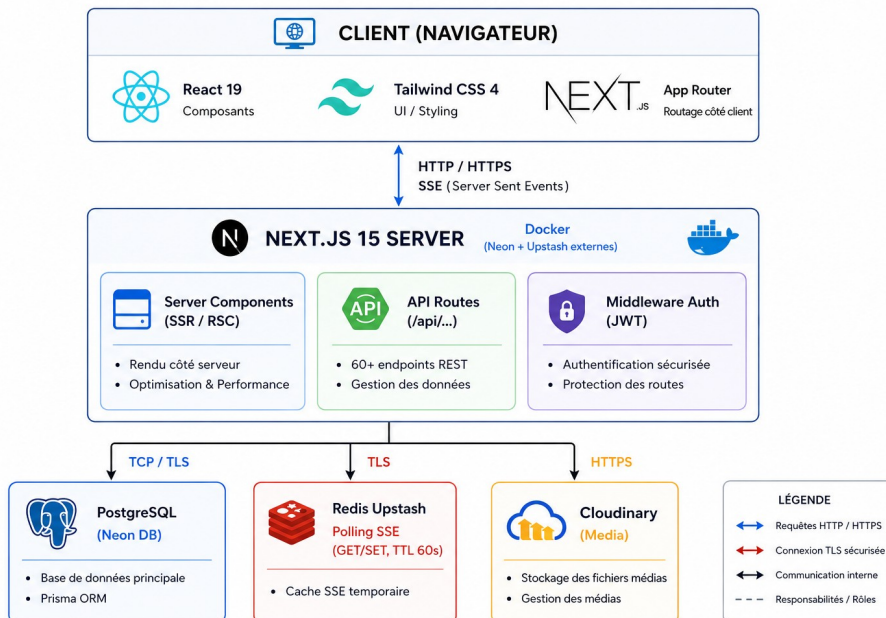
Commit — build(docker) openssl Prisma:

<https://github.com/arocchet/social-network/commit/1bf7504>

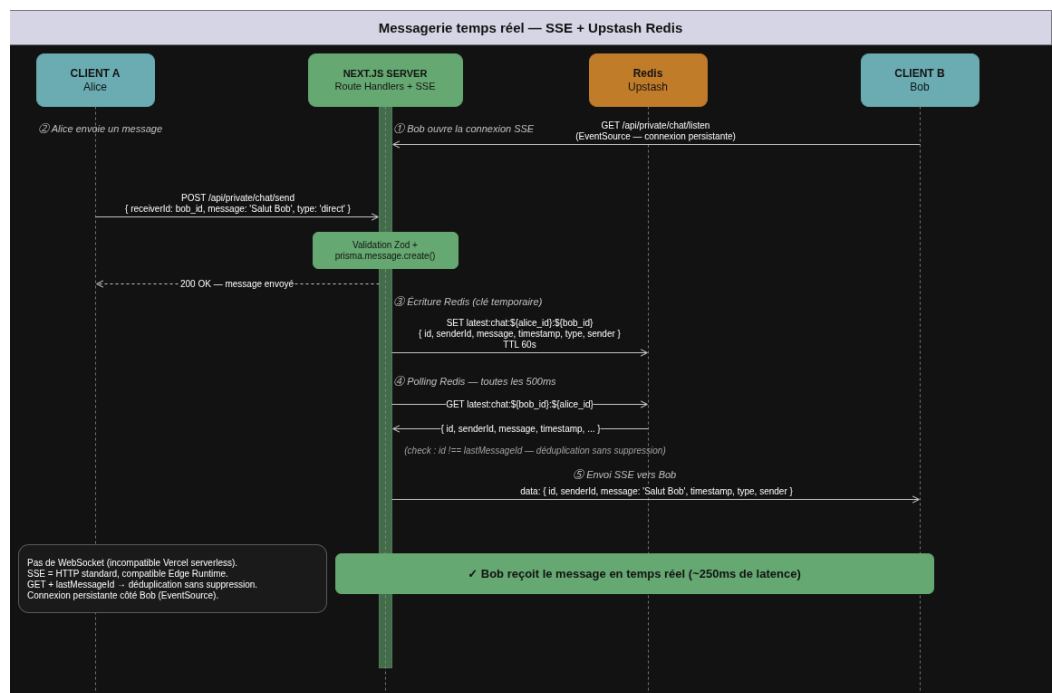
GitHub PR #73 — CLecart :

<https://github.com/arocchet/social-network/pull/73>

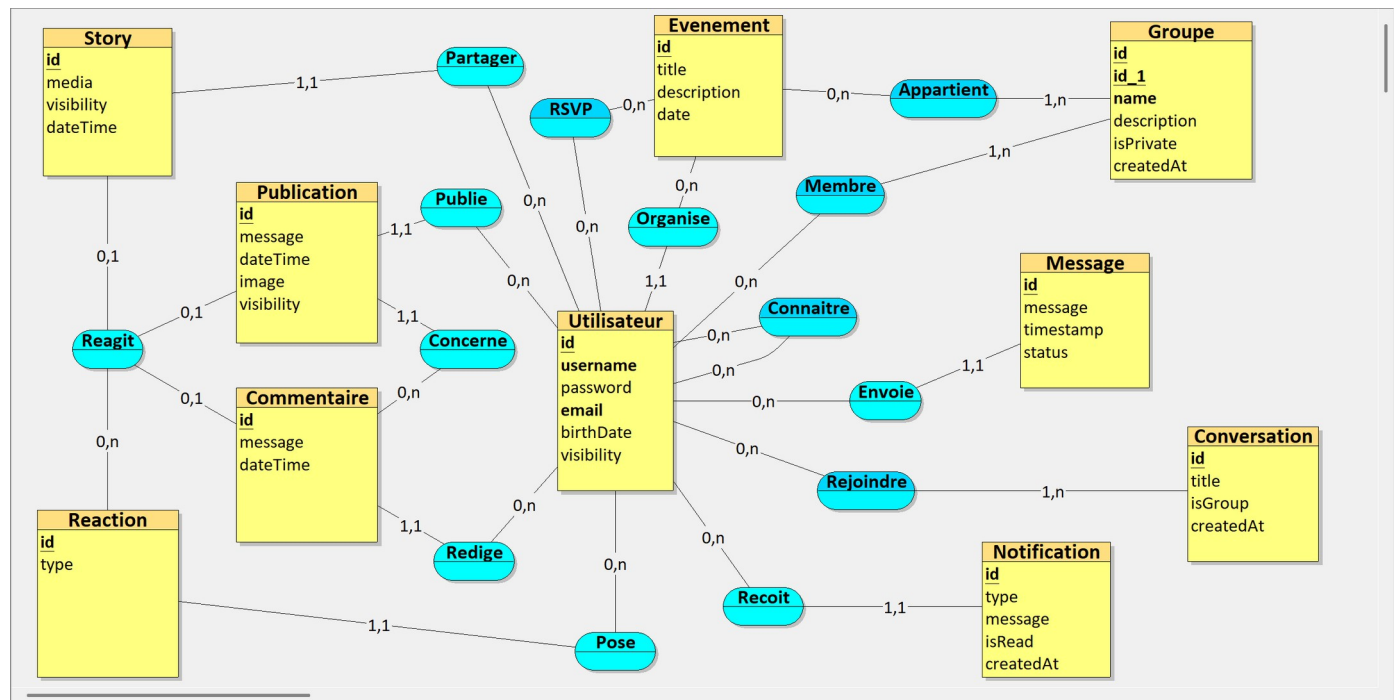
Doc — Architecture logicielle simplifiée



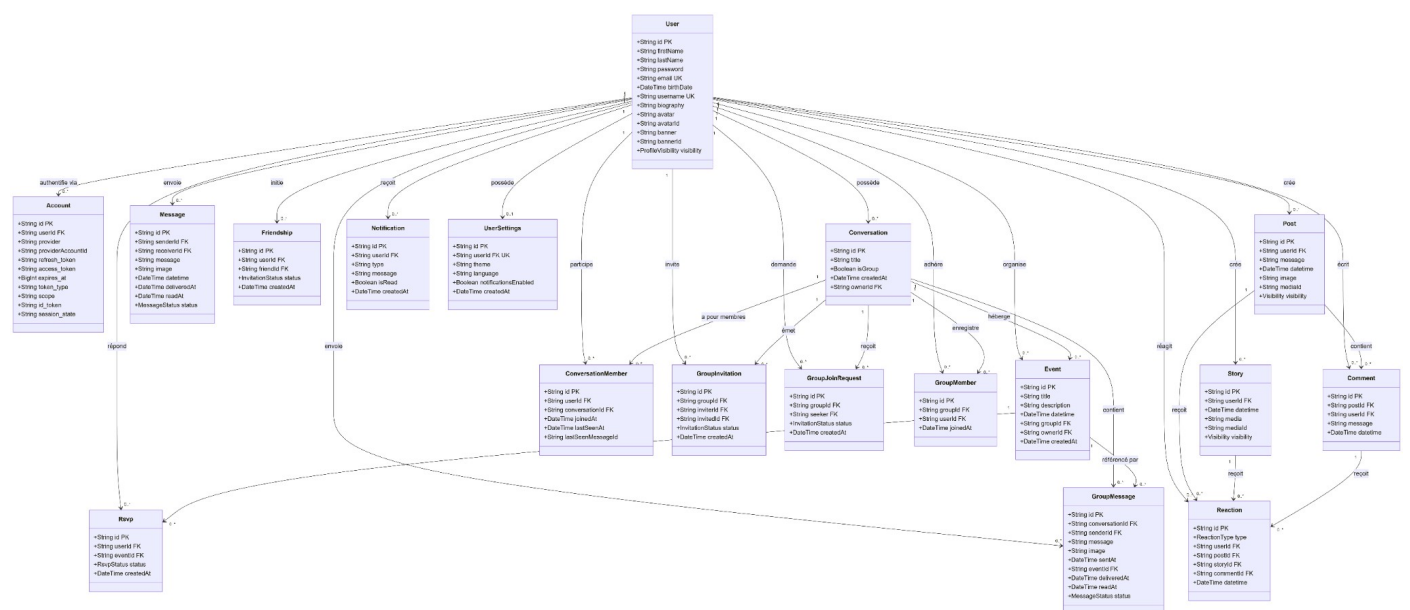
Doc — Diagramme de séquence SSE messagerie



Doc — MCD (Modèle Conceptuel de Données)

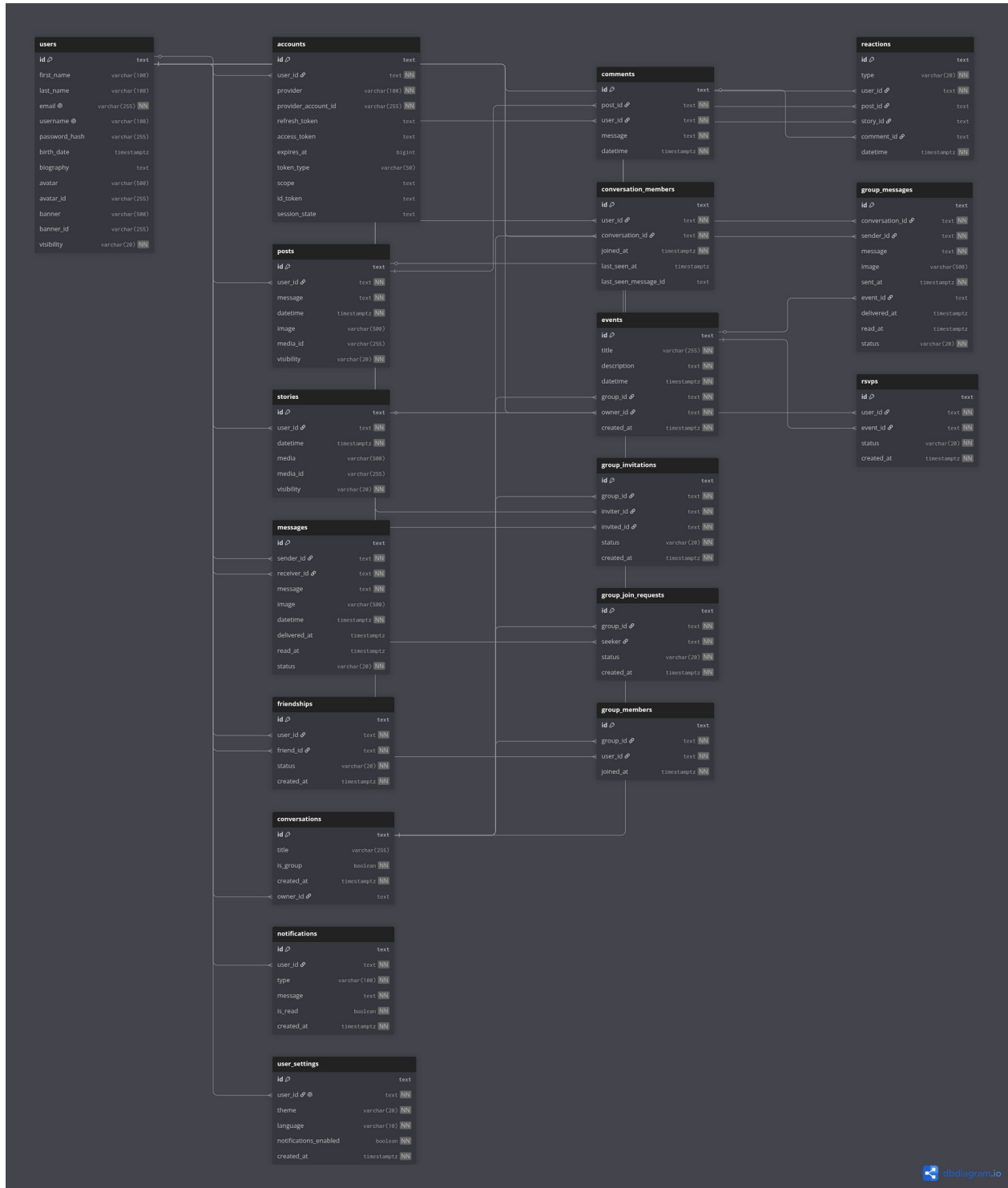


Doc — MLD (Modèle Logique de Données)



DOSSIER PROFESSIONNEL (DP)

Doc — MPD (Modèle Physique de données)



dbdiagram.io

ANNEXES

(Si le RC le prévoit)